[Bug Report] RSD Processor Does Not Handle State Changes in Floating-Point Context Status Flag

Metainformation

JVN-ID: PENDING

Contact: Feel free to reach out for more details to jv.rajendran@tamu.edu

Discoverers: Chen Chen, Jayden Koh, Stephen Muttathil, Jeyavijayan Rajendran, Texas A&M University; Ahmad-Reza Sadeghi, Technische Universitat Darmstadt;

References:
https://github.com/rsd-devel/rsd/issues/104

Proposed CVSS:
CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: **9.3 / Critical**

Macro vector: 000200
Exploitability: High
Complexity: High
Vulnerable system: High
Subsequent system: Low
Exploitation: High
Security requirements: High

Relevant CWEs:
CWE-440: Expected Behavior Violation
CWE-1281: Sequence of Processor Instructions Leads to Unexpected Behavior

Vulnerability Information

Vulnerability type: This vulnerability has implications for incorrect privilege-level transitions and context switching on the RSD processor.

Vendors of the product: RSD is maintained by Ryota Shioya, The University of Tokyo; Koji Inoue, Kyushu University, and the National Institute of Informatics (Japan), Masahiro Goshima.

Product:
RISC-V Out-Of-Order Superscalar Processor (RSD)
https://github.com/rsd-devel/rsd
commit: 7b65f6b

Has the vendor confirmed or acknowledged the vulnerability? No

Attack Type: Remote

Impact: Escalation of Privileges, Denial of Service, Information Disclosure

Affected Components: Software programs utilizing floating-point registers on the RSD processor.

Attack Vectors: Any software program that contains write operations to floating-point registers.

Description of the vulnerability: Changes to the FS field in the mstatus register are not reflected in the SD flag; specifically, setting FS to “Dirty” does not cause the SD flag to be set.

Additional Information:

This vulnerability occurs in the handling of floating-point registers by the RISC-V Out-Of-Order Superscalar Processor (RSD), which allows arbitrary processes to perform floating-point operations with data outside of the process’s context and without the processor’s awareness. The GitHub issue describing the vulnerability can be found here.

Technical Information

This vulnerability enables unprivileged users to gain improper access to floating-point resources through corrupted states created during context switching, cross-process leakage, and improper register saving and restoring.

Background

According to the RISC-V manual:

> "The SD bit is a read-only bit that summarizes whether either the FS, VS, or XS fields signal the presence of some dirty state that will require saving extended user context to memory."

RSD GitHub Commit: `7b65f6b`

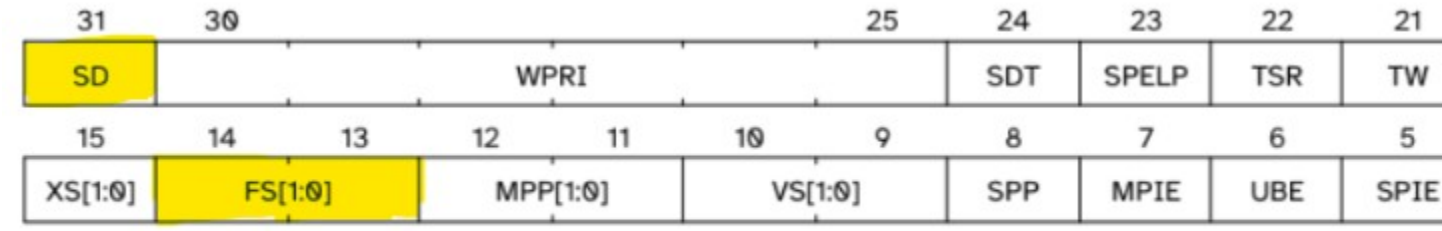


Figure 7. Machine-mode status (`mstatus`) registe

The specifications for the mstatus register indicate that the FS flag is located in `mstatus[14:13]` and the SD flag is in `mstatus[31]`.

Table 11. Encoding of FS[1:0], VS[1:0], and XS[1:0] status fields

Status	FS and VS Meaning	XS Meaning
0	Off	All off
1	Initial	None dirty or clean, some on
2	Clean	None dirty, some clean
3	Dirty	Some dirty

The State Dirty (SD) flag in the mstatus register should be modified to correctly reflect the state of other context flags, namely the Floating-Point State (FS) flag. When none of the FS, XS, and VS flags are “Dirty”, the SD flag should be Off, and when at least one of the XS, FS, or VS flags is On, the SD flag should be On.

In an example program (provided below as exploit_folder.zip), an attempt is made to write the bits “11” to the FS flag, manually setting FS to “Dirty”.

```
1af4:       300022f3                csrr    t0,mstatus
1af8:       10000337                lui     t1,0x10000
1afc:       9ff30313                addi    t1,t1,-1537
1b00:       0062f2b3                and     t0,t0,t1
1b04:       00006337                lui     t1,0x6
1b08:       0062e2b3                or      t0,t0,t1
1b0c:       30029073                csrw    mstatus,t0
1b10:       00000013                nop
1b14:       00000013                nop
1b18:       00000013                nop
```

Figure 1: *Disassembly of the exploit program with relevant instructions.*

The instruction at 0x1b0c writes “0x6000” to mstatus, setting the FS flag to “Dirty”.

Expected Behavior

If the FS flag is set to “Dirty,” then the SD flag needs to be set to On.

Actual Behavior

Below is the waveform describing the processor not properly handling the change in the FS state. The mstatus register is manually set to “0x6000,” which sets the FS flag to “Dirty”; however, the 31st bit, State Dirty (SD), does not reflect this change by changing from 0 to 1. This shows that the SD bit is being improperly handled, as it does not reflect the correct state of the other context statuses.

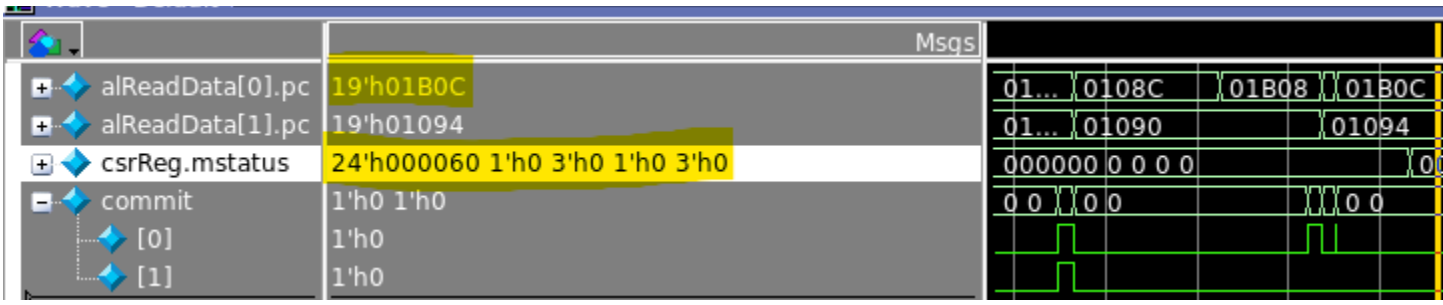


Figure 2: *Waveform of the RSD processor setting FS flag to “Dirty” but not updating SD to “On”.*

Steps to Replicate

1. Compiled C code to hex executable. The code used in the screenshot has been attached below.

exploit_folder.zip

2. Run the executable with this line:

vsim ../Project/ModelSim/Main.TestMain +MAX_TEST_CYCLES=50000 +TEST_CODE=exploit_folder/ +ENABLE_PC_GOAL=1 +SHOW_SERIAL_OUT=1 -suppress 8233 +autofindloop +RSD_LOG_FILE=RSD.log +DEBUG_LOG_FILE=Debug.log +REG_CSV_FILE=Register.csv

Proof-of-Concept Exploit

1. In Group 1, set the FS flag in mstatus to "Dirty" (11), meaning a floating-point register has been written to in this context. This should also set the SD flag in mstatus to “On” (1), meaning either FS has been set to “Dirty”.
2. In Group 2, NOP instructions allow the processor enough time to commit the instructions.
3. In Group 3, an instruction checks the mstatus register again.

```
// GROUP 1: set FS in mstatus to 01 initial
asm volatile ( "CSRR t0, mstatus" );
asm volatile ( "LI t1, 0xFFFF9FF" );
asm volatile ( "AND t0, t0, t1" ); // clear FS bits
asm volatile ( "LI t1, 0x6000" );
asm volatile ( "OR t0, t0, t1" ); // set FS to Dirty (11)
asm volatile ( "CSRW mstatus, t0" );

// GROUP 2: NOP instructions allow the processor time to commit
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3: read mstatus again
asm volatile ( "CSRR t2, mstatus" );
```